

# C++ Standard Library Issues to be moved in Cologne

Doc. no. P1724R0

Date: Revised 2019-06-17 at 05:09:46 UTC

Project: Programming Language C++

Reply to: Marshall Clow <[lwgchair@gmail.com](mailto:lwgchair@gmail.com)>

## Ready Issues

---

[2899](#)<sup>(1)</sup>. `is_(nothrow_)move_constructible` and `tuple`, `optional` and `unique_ptr`

**Section:** 20.5 [[tuple](#)], 20.6 [[optional](#)], 20.11.1.2.1 [[unique\\_ptr.single\\_ctor](#)] **Status:** [Ready](#) **Submitter:** United States **Opened:** 2017-02-03 **Last modified:** 2019-02-26

**Priority:** 2

**View all other** [issues](#) in [[tuple](#)].

**View all issues with** [Ready](#) status.

**Discussion:**

**Addresses US 110**

The move constructors for `tuple`, `optional`, and `unique_ptr` should return false for `is_(nothrow_)move_constructible_v<TYPE>` when their corresponding *Requires* clauses are not satisfied, as there are now several library clauses that are defined in terms of these traits. The same concern applies to the move-assignment operator. Note that `pair` and `variant` already satisfy this constraint.

[2017-02-26, Scott Schurr provides wording]

[ 2017-06-27 P2 after 5 positive votes on `c++std-lib`. ]

[2016-07, Toronto Thursday night issues processing]

The description doesn't match the resolution; Alisdair to investigate. Status to Open

**Previous resolution [SUPERSEDED]:**

This wording is relative to [N4640](#).

1. Modify 20.5.3 [[tuple.tuple](#)] as indicated:

```
// 20.5.3.1, tuple construction
EXPLICIT constexpr tuple();
EXPLICIT constexpr tuple(const Types&...); // only if sizeof...(Types) >= 1
template <class... UTypes>
EXPLICIT constexpr tuple(UTypes&&...) noexcept(see below); // only if sizeof...(Types) >= 1

tuple(const tuple&) = default;
tuple(tuple&&) = default;

template <class... UTypes>
EXPLICIT constexpr tuple(const tuple<UTypes...>&);
template <class... UTypes>
EXPLICIT constexpr tuple(tuple<UTypes...>&&) noexcept(see below);

template <class U1, class U2>
EXPLICIT constexpr tuple(const pair<U1, U2>&); // only if sizeof...(Types) == 2
template <class U1, class U2>
EXPLICIT constexpr tuple(pair<U1, U2>&&) noexcept(see below); // only if sizeof...(Types) == 2

[...]
```

```
// 20.5.3.2, tuple assignment
tuple& operator=(const tuple&);
tuple& operator=(tuple&&) noexcept(see below);

template <class... UTypes>
tuple& operator=(const tuple<UTypes...>&);
template <class... UTypes>
tuple& operator=(tuple<UTypes...>&&) noexcept(see below);

template <class U1, class U2>
tuple& operator=(const pair<U1, U2>&); // only if sizeof...(Types) == 2
template <class U1, class U2>
tuple& operator=(pair<U1, U2>&&) noexcept(see below); // only if sizeof...(Types) == 2
```

2. Modify 20.5.3.1 [[tuple.cnstr](#)] as indicated:

```
template <class... UTypes> EXPLICIT constexpr tuple(UTypes&&... u) noexcept(see below);
```

-8- *Effects:* Initializes the elements in the tuple with the corresponding value in `std::forward<UTypes>(u)`.

-9- *Remarks:* This constructor shall not participate in overload resolution unless `sizeof...(Types) == sizeof...(UTypes)` and `sizeof...(Types) >= 1` and `is_constructible_v<Ti, Ui&&>` is true for all *i*. The constructor is explicit if and only if `is_convertible_v<Ui&&, Ti>` is false for at least one *i*. The expression inside `noexcept` is equivalent to the logical AND of the following expressions:

`is_nothrow_constructible_v<Ti, Ui&&>`

where  $T_i$  is the  $i^{\text{th}}$  type in `Types`, and  $U_i$  is the  $i^{\text{th}}$  type in `UTypes`.

[...]

```
template <class... UTypes> EXPLICIT constexpr tuple(tuple<UTypes...>&& u) noexcept(see below);
```

-16- *Effects:* For all *i*, initializes the  $i^{\text{th}}$  element of `*this` with `std::forward<Ui>(get<i>(u))`.

-17- *Remarks:* This constructor shall not participate in overload resolution unless

[...]

The constructor is explicit if and only if `is_convertible_v<Ui&&, Ti>` is false for at least one *i*. The expression inside `noexcept` is equivalent to the logical AND of the following expressions:

`is_nothrow_constructible_v<Ti, Ui&&>`

where  $T_i$  is the  $i^{\text{th}}$  type in `Types`, and  $U_i$  is the  $i^{\text{th}}$  type in `UTypes`.

[...]

```
template <class U1, class U2> EXPLICIT constexpr tuple(pair<U1, U2>&& u) noexcept(see below);
```

-21- *Effects:* Initializes the first element with `std::forward<U1>(u.first)` and the second element with `std::forward<U2>(u.second)`.

-22- *Remarks:* This constructor shall not participate in overload resolution unless `sizeof...(Types) == 2`, `is_constructible_v<T0, U1&&>` is true and `is_constructible_v<T1, U2&&>` is true.

-23- The constructor is explicit if and only if `is_convertible_v<U1&&, T0>` is false or `is_convertible_v<U2&&, T1>` is false. The expression inside `noexcept` is equivalent to:

`is_nothrow_constructible_v<T0, U1&&> && is_nothrow_constructible_v<T1, U2&&>`

3. Modify 20.5.3.2 [tuple.assign] as indicated:

```
template <class... UTypes> tuple& operator=(tuple<UTypes...>&& u) noexcept(see below);
```

-12- *Effects:* For all *i*, assigns `std::forward<Ui>(get<i>(u))` to `get<i>(*this)`.

-13- *Remarks:* This operator shall not participate in overload resolution unless `is_assignable_v<Ti&, Ui&&> == true` for all *i* and `sizeof...(Types) == sizeof...(UTypes)`. The expression inside `noexcept` is equivalent to the logical AND of the following expressions:

`is_nothrow_assignable_v<Ti&, Ui&&>`

where  $T_i$  is the  $i^{\text{th}}$  type in `Types`, and  $U_i$  is the  $i^{\text{th}}$  type in `UTypes`.

-14- *Returns:* `*this`.

[...]

```
template <class U1, class U2> tuple& operator=(pair<U1, U2>&& u) noexcept(see below);
```

-18- *Effects:* Assigns `std::forward<U1>(u.first)` to the first element of `*this` and `std::forward<U2>(u.second)` to the second element of `*this`.

-19- *Remarks:* This operator shall not participate in overload resolution unless `sizeof...(Types) == 2` and `is_assignable_v<T0&, U1&&>` is true for the first type  $T_0$  in `Types` and `is_assignable_v<T1&, U2&&>` is true for the second type  $T_1$  in `Types`. The expression inside `noexcept` is equivalent to:

`is_nothrow_assignable_v<T0&, U1&&> && is_nothrow_assignable_v<T1&, U2&&>`

-20- *Returns:* `*this`.

[2019-02-14; Jonathan comments and provides revised wording]

The suggested change was already made to `std::optional` by LWG 2756. The current P/R for 2899 doesn't resolve the issue for `std::tuple` or `std::unique_ptr`. I hope the following alternative does.

[2019-02; Kona Wednesday night issue processing]

Status to Ready

**Proposed resolution:**

This wording is relative to [N4800](#).

1. Modify 20.5.3.1 [\[tuple.cnstr\]](#) as indicated:

```
tuple(tuple&& u) = default;
```

-13- ~~Requires~~ **Constraints**: `is_move_constructible_v<Ti>` is true for all *i*.

-14- *Effects*: For all *i*, initializes the *i*<sup>th</sup> element of `*this` with `std::forward<Ti>(get<i>(u))`.

2. Modify 20.11.1.2.1 [\[unique.ptr.single.ctor\]](#) as indicated:

```
unique_ptr(unique_ptr&& u) noexcept;
```

~~?- Constraints~~: `is_move_constructible_v<D>` is true.

-15- *Requires*: If *D* is not a reference type, *D* shall satisfy the *Cpp17MoveConstructible* requirements (Table 26). Construction of the deleter from an rvalue of type *D* shall not throw an exception.

[...]

3. Modify 20.11.1.2.3 [\[unique.ptr.single.asgn\]](#) as indicated:

```
unique_ptr& operator=(unique_ptr&& u) noexcept;
```

~~?- Constraints~~: `is_move_assignable_v<D>` is true.

-1- *Requires*: If *D* is not a reference type, *D* shall satisfy the *Cpp17MoveAssignable* requirements (Table 28) and assignment of the deleter from an rvalue of type *D* shall not throw an exception. Otherwise, *D* is a reference type; `remove_reference_t<D>` shall satisfy the *Cpp17CopyAssignable* requirements and assignment of the deleter from an lvalue of type *D* shall not throw an exception.

-2- *Effects*: Calls `reset(u.release())` followed by `get_deleter() = std::forward<D>(u.get_deleter())`.

[...]

---

### [3055<sup>\(i\)</sup>](#). `path::operator+=(single-character)` misspecified

**Section:** 29.11.7.4.4 [\[fs.path.concat\]](#) **Status:** [Ready](#) **Submitter:** Tim Song **Opened:** 2018-01-24 **Last modified:** 2019-02-26

**Priority:** 3

**View all other [issues](#)** in `[fs.path.concat]`.

**View all issues with [Ready](#) status.**

#### Discussion:

29.11.7.4.4 [\[fs.path.concat\]](#) uses the expression `path(x).native()` to specify the effects of concatenating a single character *x*. However, there is no `path` constructor taking a single character.

[2018-06-18 after reflector discussion]

Priority set to 3

[2018-10-12 Tim updates PR to avoid suggesting the creation of a temporary path.]

**Previous resolution [SUPERSEDED]:**

This wording is relative to [N4713](#).

1. Modify 29.11.7.4.4 [\[fs.path.concat\]](#) as indicated:

```
path& operator+=(const path& x);
path& operator+=(const string_type& x);
path& operator+=(basic_string_view<value_type> x);
path& operator+=(const value_type* x);
path& operator+=(value_type x);
template<class Source>
    path& operator+=(const Source& x);
template<class ECharT>
path& operator+=(ECharT x);
template<class Source>
    path& concat(const Source& x);
```

-1- *Effects*: [...]

-2- *Returns*: `*this`.

**path& operator+=(value\_type x);**  
**template<class ECharT>**  
**path& operator+=(ECharT x);**

**-?- Effects**: Equivalent to: `return *this += path(&x, &x + 1);`.

[2019-02; Kona Wednesday night issue processing]

Status to Ready

#### Proposed resolution:

This wording is relative to [N4762](#).

1. Modify 29.11.7.4.4 [\[fs.path.concat\]](#) as indicated:

```
path& operator+=(const path& x);
path& operator+=(const string_type& x);
path& operator+=(basic_string_view<value_type> x);
path& operator+=(const value_type* x);
path& operator+=(value_type x);
template<class Source>
    path& operator+=(const Source& x);
template<class EcharT>
path& operator+=(EcharT x);
template<class Source>
    path& concat(const Source& x);
```

-1- *Effects*: [...]

-2- *Returns*: \*this.

```
path& operator+=(value_type x);
template<class EcharT>
    path& operator+=(EcharT x);
```

-?- *Effects*: Equivalent to: `return *this += basic_string_view(&x, 1);`.

---

### [3158](#)<sup>(i)</sup>. `tuple(allocator_arg_t, const Alloc&)` should be conditionally explicit

**Section:** 20.5.3.1 [\[tuple.cnstr\]](#) **Status:** [Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2018-08-21 **Last modified:** 2019-02-26

**Priority:** 3

**View other [active issues](#)** in [\[tuple.cnstr\]](#).

**View all other [issues](#)** in [\[tuple.cnstr\]](#).

**View all issues with [Ready](#) status.**

#### Discussion:

`std::tuple`'s allocator-extended constructors say "*Effects*: Equivalent to the preceding constructors except that each element is constructed with uses-allocator construction". That's not true for the first one, as shown by:

```
#include <tuple>

struct X { explicit X() { } };

std::tuple<X> f() { return {}; }
std::tuple<X> g() { return { std::allocator_arg, std::allocator<int>{} }; }
```

The function `f()` doesn't compile because of the explicit constructor, but `g()` does, despite using the same constructor for `X`. The conditional explicit-ness is not equivalent.

Also, the editor requested that we change "implicitly default-constructible" to use words that mean something. He suggested "copy-list-initializable from an empty list".

[2018-09 Reflector prioritization]

Set Priority to 3

[2019-02; Kona Wednesday night issue processing]

Status to Ready

#### Proposed resolution:

This wording is relative to [N4762](#).

1. Modify 20.5.3 [\[tuple.tuple\]](#), class template `tuple` synopsis, as indicated:

```
[...]
// allocator-extended constructors
template<class Alloc>
    explicit\(see below\) tuple(allocator_arg_t, const Alloc& a);
template<class Alloc>
    explicit(see below) tuple(allocator_arg_t, const Alloc& a, const Types&...);
[...]
```

2. Modify 20.5.3.1 [\[tuple.cnstr\]](#), as indicated:

```
explicit(see below) constexpr tuple();
```

-5- *Effects*: Value-initializes each element.

-6- *Remarks:* This constructor shall not participate in overload resolution unless `is_default_constructible_v<Ti>` is true for all *i*.  
[*Note:* This behavior can be implemented by a constructor template with default template arguments. — *end note*] The expression inside `explicit` evaluates to true if and only if `Ti` is not ~~implicitly default-constructible~~ `copy-list-initializable from an empty list` for at least one *i*. [*Note:* This behavior can be implemented with a trait that checks whether a `const Ti&` can be initialized with `{}`. — *end note*]

[...]

```
template<class Alloc>
    explicit(see below) tuple(allocator_arg_t, const Alloc& a);
template<class Alloc>
    explicit(see below) tuple(allocator_arg_t, const Alloc& a, const Types&...);
[...]
```

```
template<class Alloc, class U1, class U2>
    explicit(see below) tuple(allocator_arg_t, const Alloc& a, pair<U1, U2>&&);
```

-25- *Requires:* Alloc shall satisfy the `Cpp17Allocator` requirements (Table 33).

[...]

## 3169<sup>(1)</sup>. ranges permutation generators discard useful information

**Section:** 25.7.12 [alg.permutation.generators] **Status:** [Ready](#) **Submitter:** Casey Carter **Opened:** 2018-11-26 **Last modified:** 2019-02-26

**Priority:** 0

**View all issues with** [Ready](#) status.

### Discussion:

In the Ranges design, algorithms that necessarily traverse an entire range - consequently discovering the end iterator value - return that iterator value unless the algorithm's sole purpose is to return a derived value, for example, `ranges::count`, `ranges::next_permutation` and `ranges::prev_permutation` necessarily traverse the entirety of their range argument, but are currently specified to discard the end iterator value and return only a `bool` indicating whether they found a next (respectively previous) permutation or "reset" the range to the first (respectively last) permutation. They should instead return an aggregate composed of both that `bool` and the end iterator value to be consistent with the other range algorithms.

[2019-01-22; Daniel comments and updates wording]

During the reflector discussion it had been noticed that an additional update of 25.2 [algorithms.requirements] p.16 is necessary for the new type `next_permutation_result` and two missing occurrences of `iterator_t<>` where added. The proposed wording has been updated.

[2019-02-02 Priority to 0 and Status to Tentatively Ready after five positive votes on the reflector.]

### Previous resolution [SUPERSEDED]:

1. Modify 25.2 [algorithms.requirements] as follows:

-16- The class templates `binary_transform_result`, `for_each_result`, `minmax_result`, `mismatch_result`, `next_permutation_result`, `copy_result`, and `partition_copy_result` have the template parameters, data members, and special members specified above. They have no base classes or members other than those specified.

2. Modify 25.4 [algorithm.syn] as follows:

```
// 25.7.12 [alg.permutation.generators], permutations
template<class BidirectionalIterator>
    constexpr bool next_permutation(BidirectionalIterator first,
                                    BidirectionalIterator last);
template<class BidirectionalIterator, class Compare>
    constexpr bool next_permutation(BidirectionalIterator first,
                                    BidirectionalIterator last, Compare comp);

namespace ranges {
    template<class I>
    struct next_permutation_result {
        bool found;
        I in;
    };

    template<BidirectionalIterator I, Sentinel<I> S, class Comp = ranges::less<>,
            class Proj = identity>
    requires Sortable<I, Comp, Proj>
    constexpr bool next_permutation_result<I>
        next_permutation(I first, S last, Comp comp = {}, Proj proj = {});
    template<BidirectionalRange R, class Comp = ranges::less<>,
            class Proj = identity>
    requires Sortable<iterator_t<R>, Comp, Proj>
    constexpr bool next_permutation_result<iterator_t<R>>
        next_permutation(R&& r, Comp comp = {}, Proj proj = {});
}

template<class BidirectionalIterator>
    constexpr bool prev_permutation(BidirectionalIterator first,
                                    BidirectionalIterator last);
template<class BidirectionalIterator, class Compare>
    constexpr bool prev_permutation(BidirectionalIterator first,
                                    BidirectionalIterator last, Compare comp);

namespace ranges {
```

```

template<class I>
using prev_permutation_result = next_permutation_result<I>;

template<BidirectionalIterator I, Sentinel<I> S, class Comp = ranges::less<>,
        class Proj = identity>
requires Sortable<I, Comp, Proj>
constexpr bool prev_permutation_result<I>
    prev_permutation(I first, S last, Comp comp = {}, Proj proj = {});
template<BidirectionalRange R, class Comp = ranges::less<>,
        class Proj = identity>
requires Sortable<iterator_t<R>, Comp, Proj>
constexpr bool prev_permutation_result<iterator_t<R>>
    prev_permutation(R&& r, Comp comp = {}, Proj proj = {});
}
}

```

3. Modify 25.7.12 [alg.permutation.generators] as follows:

```

template<class BidirectionalIterator>
constexpr bool next_permutation(BidirectionalIterator first,
                               BidirectionalIterator last);
template<class BidirectionalIterator, class Compare>
constexpr bool next_permutation(BidirectionalIterator first,
                               BidirectionalIterator last, Compare comp);

namespace ranges {
    template<BidirectionalIterator I, Sentinel<I> S, class Comp = ranges::less<>,
            class Proj = identity>
    requires Sortable<I, Comp, Proj>
    constexpr bool next_permutation_result<I>
        next_permutation(I first, S last, Comp comp = {}, Proj proj = {});
    template<BidirectionalRange R, class Comp = ranges::less<>,
            class Proj = identity>
    requires Sortable<iterator_t<R>, Comp, Proj>
    constexpr bool next_permutation_result<iterator_t<R>>
        next_permutation(R&& r, Comp comp = {}, Proj proj = {});
}

[...]

```

-4- Returns: Let B be true if ~~and only if~~ a next permutation was found and otherwise false. Returns:

- B for the overloads in namespace std, or
- { B, last } for the overloads in namespace ranges.

-5- Complexity: [...]

```

template<class BidirectionalIterator>
constexpr bool prev_permutation(BidirectionalIterator first,
                               BidirectionalIterator last);
template<class BidirectionalIterator, class Compare>
constexpr bool prev_permutation(BidirectionalIterator first,
                               BidirectionalIterator last, Compare comp);

namespace ranges {
    template<BidirectionalIterator I, Sentinel<I> S, class Comp = ranges::less<>,
            class Proj = identity>
    requires Sortable<I, Comp, Proj>
    constexpr bool prev_permutation_result<I>
        prev_permutation(I first, S last, Comp comp = {}, Proj proj = {});
    template<BidirectionalRange R, class Comp = ranges::less<>,
            class Proj = identity>
    requires Sortable<iterator_t<R>, Comp, Proj>
    constexpr bool prev_permutation_result<iterator_t<R>>
        prev_permutation(R&& r, Comp comp = {}, Proj proj = {});
}

[...]

```

-9- Returns: Let B be true if ~~and only if~~ a previous permutation was found and otherwise false. Returns:

- B for the overloads in namespace std, or
- { B, last } for the overloads in namespace ranges.

-10- Complexity: [...]

[2019-02-10 Tomasz comments; Casey updates the P/R and resets status to "Review."]

Shouldn't the range overloads for an algorithms return `safe_iterator_t<R>` instead of `iterator_t<R>`? Other algorithms are consistently returning the `safe_iterator_t/safe_subrange_t` in situation, when range argument is an rvalue (temporary) and returned iterator may be dangling.

[2019-02; Kona Wednesday night issue processing]

Status to Ready

**Proposed resolution:**

This wording is relative to [N4800](#).

1. Modify 25.2 [\[algorithms.requirements\]](#) as follows:

-16- The class templates `binary_transform_result`, `for_each_result`, `minmax_result`, `mismatch_result`, `next_permutation_result`, `copy_result`, and `partition_copy_result` have the template parameters, data members, and special members specified above. They have no base classes or members other than those specified.

2. Modify 25.4 [\[algorithm.syn\]](#) as follows:

```
// 25.7.12 \[alg.permutation.generators\], permutations
template<class BidirectionalIterator>
constexpr bool next_permutation(BidirectionalIterator first,
                               BidirectionalIterator last);
template<class BidirectionalIterator, class Compare>
constexpr bool next_permutation(BidirectionalIterator first,
                               BidirectionalIterator last, Compare comp);

namespace ranges {
    template<class I>
    struct next_permutation_result {
        bool found;
        I in;
    };

    template<BidirectionalIterator I, Sentinel<I> S, class Comp = ranges::less<>,
            class Proj = identity>
    requires Sortable<I, Comp, Proj>
    constexpr bool next_permutation_result<I>
        next_permutation(I first, S last, Comp comp = {}, Proj proj = {});
    template<BidirectionalRange R, class Comp = ranges::less<>,
            class Proj = identity>
    requires Sortable<iterator_t<R>, Comp, Proj>
    constexpr bool next_permutation_result<safe_iterator t<R>>
        next_permutation(R&& r, Comp comp = {}, Proj proj = {});
}

template<class BidirectionalIterator>
constexpr bool prev_permutation(BidirectionalIterator first,
                               BidirectionalIterator last);
template<class BidirectionalIterator, class Compare>
constexpr bool prev_permutation(BidirectionalIterator first,
                               BidirectionalIterator last, Compare comp);

namespace ranges {
    template<class I>
    using prev_permutation_result = next_permutation_result<I>;

    template<BidirectionalIterator I, Sentinel<I> S, class Comp = ranges::less<>,
            class Proj = identity>
    requires Sortable<I, Comp, Proj>
    constexpr bool prev_permutation_result<I>
        prev_permutation(I first, S last, Comp comp = {}, Proj proj = {});
    template<BidirectionalRange R, class Comp = ranges::less<>,
            class Proj = identity>
    requires Sortable<iterator_t<R>, Comp, Proj>
    constexpr bool prev_permutation_result<safe_iterator t<R>>
        prev_permutation(R&& r, Comp comp = {}, Proj proj = {});
}
}
```

3. Modify 25.7.12 [\[alg.permutation.generators\]](#) as follows:

```
template<class BidirectionalIterator>
constexpr bool next_permutation(BidirectionalIterator first,
                               BidirectionalIterator last);
template<class BidirectionalIterator, class Compare>
constexpr bool next_permutation(BidirectionalIterator first,
                               BidirectionalIterator last, Compare comp);

namespace ranges {
    template<BidirectionalIterator I, Sentinel<I> S, class Comp = ranges::less<>,
            class Proj = identity>
    requires Sortable<I, Comp, Proj>
    constexpr bool next_permutation_result<I>
        next_permutation(I first, S last, Comp comp = {}, Proj proj = {});
    template<BidirectionalRange R, class Comp = ranges::less<>,
            class Proj = identity>
    requires Sortable<iterator_t<R>, Comp, Proj>
    constexpr bool next_permutation_result<safe_iterator t<R>>
        next_permutation(R&& r, Comp comp = {}, Proj proj = {});
}

[...]
```

-4- Returns: `Let B be` true if ~~and only if~~ a next permutation was found ~~and otherwise false~~. Returns:

- `B` for the overloads in namespace `std`, or
- `{ B, last }` for the overloads in namespace `ranges`.

-5- Complexity: [...]

```
template<class BidirectionalIterator>
constexpr bool prev_permutation(BidirectionalIterator first,
                               BidirectionalIterator last);
template<class BidirectionalIterator, class Compare>
```

```
constexpr bool prev_permutation(BidirectionalIterator first,
                               BidirectionalIterator last, Compare comp);

namespace ranges {
    template<BidirectionalIterator I, Sentinel<I> S, class Comp = ranges::less<>,
            class Proj = identity>
        requires Sortable<I, Comp, Proj>
        constexpr bool prev_permutation_result<I>
            prev_permutation(I first, S last, Comp comp = {}, Proj proj = {});
    template<BidirectionalRange R, class Comp = ranges::less<>,
            class Proj = identity>
        requires Sortable<iterator_t<R>, Comp, Proj>
        constexpr bool prev_permutation_result<safe_iterator_t<R>>
            prev_permutation(R&& r, Comp comp = {}, Proj proj = {});
}
```

[...]

-9- Returns: **Let B be** true if **and only if** a previous permutation was found **and otherwise false**. Returns:

- **B for the overloads in namespace std, or**
- **{ B, last } for the overloads in namespace ranges.**

-10- Complexity: [...]

## 3183<sup>(4)</sup>. Normative permission to specialize Ranges variable templates

Section: 23.3.4.8 [iterator.concept.sizedsentinel] Status: [Tentatively Ready](#) Submitter: Casey Carter Opened: 2019-01-14 Last modified: 2019-02-26

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

[P0896R4 "The One Ranges Proposal"](#) added boolean variable templates `std::disable_sized_sentinel` and `std::ranges::disable_sized_range` which users are intended to specialize to false for program-defined Iterator-Sentinel pairs / Range types which meet the syntax but do not model the semantics of the `SizedSentinel` / `SizedRange` concepts, respectively. Specializing these traits allows the use of such types with the library which would otherwise treat them as if they model `SizedSentinel` / `SizedRange`. The wording in [P0896R4](#) failed, however, to provide normative permission to specialize these variable templates as is required by 16.5.4.2.1 [namespace.std] after the application of [P0551R3](#).

Furthermore, 16.5.4.2.1 [namespace.std] notably does not require that program-defined specializations of standard library variable templates meet the requirements on the primary template (as is the case for class templates) or indeed any requirements. [P0896R4](#) also added the `enable_view` variable template which is used to explicitly opt in or out of the view concept 24.4.4 [range.view] when the default chosen by the heuristic is incorrect. [P0896R4](#) did include normative permission to specialize `enable_view`, but the wording does not place sufficient requirements on such user specializations so as to make them usable by the view concept definition. Specializations must be required to be usable as constant expressions of type `bool` to avoid hard errors in the concept.

[2019-02-03 Priority to 0 and Status to Tentatively Ready after five positive votes on the reflector.]

Proposed resolution:

This wording is relative to [N4791](#).

[Drafting Note: This wording uses the recently-defined core language term "usable in constant expressions" from 7.7 [expr.const] paragraph 3 which may be unfamiliar to reviewers.]

1. Change 23.3.4.8 [iterator.concept.sizedsentinel] as follows:

[...]

(2.2) — If `-N` is representable by `iter_difference_t<I>`, then `i - s` is well-defined and equals `-N`.

**?- Pursuant to 16.5.4.2.1 [namespace.std], users may specialize `disable_sized_sentinel` for cv-unqualified non-array object types `S` and `I` at least one of which is a program-defined type. Such specializations shall be usable in constant expressions (7.7 [expr.const]) and have type `const bool`.**

3 [Note: `disable_sized_sentinel` allows use of sentinels and iterators with the library that satisfy but do not in fact model `SizedSentinel`.—end note]

[...]

2. Add an index entry for `disable_sized_sentinel` that points to [iterator.concepts.sizedsentinel].

3. Change 24.4.3 [range.sized] as follows:

[...]

3 [Note: The complexity requirement for the evaluation of `ranges::size` is non-amortized, unlike the case for the complexity of the evaluations of `ranges::begin` and `ranges::end` in the Range concept.—end note]

**?- Pursuant to 16.5.4.2.1 [namespace.std], users may specialize `disable_sized_range` for cv-unqualified program-defined types. Such specializations shall be usable in constant expressions (7.7 [expr.const]) and have type `const bool`.**



4 [Note: `disable_sized_range` allows use of range types with the library that satisfy but do not in fact model `SizedRange`.—end note]

4. Add an index entry for `disable_sized_range` that points to 24.4.3 [range.sized].

5. Change 24.4.4 [range.view] as follows:

[...]

5 Pursuant to 16.5.4.2.1 [namespace.std], users may specialize `enable_view` to true for `cv-unqualified program-defined` types which model View, and false for types which do not. Such specializations shall be usable in constant expressions (7.7 [expr.const]) and have type `const bool`.

### 3184<sup>(i)</sup>. Inconsistencies in `bind_front` wording

**Section:** 99 [func.bind\_front] **Status:** [Tentatively Ready](#) **Submitter:** Tomasz Kamiński **Opened:** 2019-01-16 **Last modified:** 2019-02-26

**Priority:** 0

**View all issues with** [Tentatively Ready](#) status.

#### Discussion:

During the merge of the [P0356R5](#), following "oddities" of the new wording was pointed out by Jens Maurer:

1. The initialization of the state entities of the `bind_front/not_fn` is specified using formulation "`xx` initialized with the *initializer* (`yyy`)". Per author knowledge this specification is correct, however inconsistent with the other parts of the of the standard, that direct-non-list-initialization term in such context.
2. The specification of the *Mandates* element for `bind_front` uses `conjunction_v` to specify conjunction of the requirements, while corresponding element of the `not_fn` specifies it using `&&`. As `conjunction_v` implies order of evaluation that is not necessary in this case (for every valid program, all provided traits must evaluate to true), it may be replaced with usage of fold expression with operator `&&`.

[2019-01-26 Priority to 0 and Status to Tentatively Ready after five positive votes on the reflector.]

#### Proposed resolution:

This wording is relative to [N4791](#).

1. Change 99 [func.not\_fn] as indicated:

```
template<class F> unspecified not_fn(F&& f);
```

-1- In the text that follows:

(1.1) — [...]

(1.2) — [...]

(1.3) — `fd` is the target object of `g` (20.14.2 [func.def]) of type `FD` direct-non-list-initialized with ~~initialized with the *initializer*~~  
~~`std::forward<F>(f)` (9.3 [decl.init]),~~

(1.4) — [...]

2. Change 99 [func.bind\_front] as indicated:

```
template <class F, class... Args>  
unspecified bind_front(F&& f, Args&&... args);
```

-1- In the text that follows:

(1.1) — [...]

(1.2) — [...]

(1.3) — `fd` is the target object of `g` (20.14.2 [func.def]) of type `FD` direct-non-list-initialized with ~~initialized with the *initializer*~~  
~~`std::forward<F>(f)` (9.3 [decl.init]),~~

(1.4) — [...]

(1.5) — `bound_args` is a pack of bound argument entities of `g` (20.14.2 [func.def]) of types `BoundArgs...`, direct-non-list-initialized with initializers ~~(`std::forward<Args>(args)`), ..., respectively, and~~

(1.6) — [...]

-2- *Mandates*:

`is_constructible_v<FD, F> && is_move_constructible_v<FD> &&`  
`(is_constructible_v<BoundArgs, Args> && ...) && (is_move_constructible_v<BoundArgs> && ...)` ~~`conjunction_v<is_constructible`~~  
~~`is_constructible<BoundArgs, Args>..., is_move_constructible<BoundArgs>...`~~

shall be true.

### 3185<sup>(i)</sup>. Uses-allocator construction functions missing `constexpr` and `noexcept`

**Section:** 20.10.8.2 [allocator.uses.construction] **Status:** [Tentatively Ready](#) **Submitter:** Pablo Halpern **Opened:** 2019-01-29 **Last modified:** 2019-02-26

Priority: 0

View other [active issues](#) in [allocator.uses.construction].

View all other [issues](#) in [allocator.uses.construction].

View all issues with [Tentatively Ready](#) status.

#### Discussion:

The uses-allocator construction functions introduced into WP when [P0591r4](#) was accepted (Nov 2018, San Diego) should all be constexpr. All but two should also be noexcept. Getting this right is an important part of correctly adding constexpr memory allocation into the WP.

The minimal change is to add the constexpr to all of the new functions except uninitialized\_construct\_using\_allocator and noexcept to all of the overloads of uses\_allocator\_construction\_args. Optionally, we could consider adding conditional noexcept to the remaining two functions. If [p0784](#) is accepted, then also add constexpr to uninitialized\_construct\_using\_allocator.

[2019-02-12 Priority to 0 and Status to Tentatively Ready after six positive votes on the reflector.]

#### Proposed resolution:

This wording is relative to [N4800](#).

1. Change header <memory> synopsis, 20.10.2 [\[memory.syn\]](#), as indicated:

```
[...]
// 20.10.8.2 \[allocator.uses.construction\], uses-allocator construction
template <class T, class Alloc, class... Args>
constexpr auto uses_allocator_construction_args(const Alloc& alloc, Args&&... args) noexcept -> see below;
template <class T, class Alloc, class Tuple1, class Tuple2>
constexpr auto uses_allocator_construction_args(const Alloc& alloc, piecewise_construct_t,
                                               Tuple1&& x, Tuple2&& y) noexcept -> see below;

template <class T, class Alloc>
constexpr auto uses_allocator_construction_args(const Alloc& alloc) noexcept -> see below;
template <class T, class Alloc, class U, class V>
constexpr auto uses_allocator_construction_args(const Alloc& alloc, U&& u, V&& v) noexcept -> see below;
template <class T, class Alloc, class U, class V>
constexpr auto uses_allocator_construction_args(const Alloc& alloc, const pair<U,V>& pr) noexcept -> see below;
template <class T, class Alloc, class U, class V>
constexpr auto uses_allocator_construction_args(const Alloc& alloc, pair<U,V>&& pr) noexcept -> see below;
template <class T, class Alloc, class... Args>
constexpr T make_obj_using_allocator(const Alloc& alloc, Args&&... args);
template <class T, class Alloc, class... Args>
T* uninitialized_construct_using_allocator(T* p, const Alloc& alloc, Args&&... args);
[...]
```

2. Change 20.10.8.2 [\[allocator.uses.construction\]](#) as indicated:

```
template <class T, class Alloc, class... Args>
constexpr auto uses_allocator_construction_args(const Alloc& alloc, Args&&... args) noexcept -> see below;

[...]

template <class T, class Alloc, class Tuple1, class Tuple2>
constexpr auto uses_allocator_construction_args(const Alloc& alloc, piecewise_construct_t,
                                               Tuple1&& x, Tuple2&& y) noexcept -> see below;

[...]

template <class T, class Alloc>
constexpr auto uses_allocator_construction_args(const Alloc& alloc) noexcept -> see below;

[...]

template <class T, class Alloc, class U, class V>
constexpr auto uses_allocator_construction_args(const Alloc& alloc, U&& u, V&& v) noexcept -> see below;

[...]

template <class T, class Alloc, class U, class V>
constexpr auto uses_allocator_construction_args(const Alloc& alloc, const pair<U,V>& pr) noexcept -> see below;

[...]

template <class T, class Alloc, class U, class V>
constexpr auto uses_allocator_construction_args(const Alloc& alloc, pair<U,V>&& pr) noexcept -> see below;

[...]

template <class T, class Alloc, class... Args>
constexpr T make_obj_using_allocator(const Alloc& alloc, Args&&... args);

[...]

template <class T, class Alloc, class... Args>
T* uninitialized_construct_using_allocator(T* p, const Alloc& alloc, Args&&... args);

[...]
```

## [3186<sup>\(f\)</sup>](#). ranges removal, partition, and partial\_sort\_copy algorithms discard useful information

**Section:** 25.6.8 [\[alg.remove\]](#), 25.6.9 [\[alg.unique\]](#), 25.7.1.4 [\[partial.sort.copy\]](#), 25.7.4 [\[alg.partitions\]](#) **Status:** [Ready](#) **Submitter:** Tomasz Kamiński **Opened:** 2019-02-05  
**Last modified:** 2019-02-26

**Priority:** 1

View all other [issues](#) in [\[alg.remove\]](#).

View all issues with [Ready](#) status.

### Discussion:

This is direct follow-up on the LWG issue [3169](#), that proposed to change additional algorithms that drop the iterator value equal to sentinel, that needs to be always computed. These set include removal (remove, remove\_if, and unique), partition (partition, stable\_partition), and partial\_sort\_copy.

For removal algorithms, the end of "not-erased" objects, and the "end-of-range" iterator forms a valid range of objects with unspecified value (that can be overwritten), thus we propose to return subrange.

For partition algorithms, the end of "true" object, and the "end-of-range" iterator forms a valid range of objects for which predicate returns "false", thus we propose to return subrange.

For partial\_sort\_copy we propose to return partial\_sort\_copy\_result as an alias to copy\_result to match other copy algorithms.

*[2019-02-12; Tomasz comments and improves proposed wording]*

Proposed wording is updated to incorporate wording comments from Casey Carter:

- We don't need to repeat the definition of partial\_sort\_copy\_result in 25.7.1.4 [\[partial.sort.copy\]](#); the single definition in the synopsis is sufficient.
- *e* is a potentially confusing choice of placeholder name for the end of the output range, given that we use a placeholder *E* for predicates in the algorithm specifications.

The placeholder *e* is replaced with *j* that seems not to be used in the specification of above algorithms.

*[2019-02 Priority set to 1 after reflector discussion]*

*[2019-02; Kona Wednesday night issue processing]*

Status to Ready

### Proposed resolution:

This wording is relative to [N4800](#).

1. Change header `<algorithm>` synopsis, 25.4 [\[algorithm.syn\]](#), as indicated:

```
[...]
//25.6.8 \[alg.remove\], remove
[...]
namespace ranges {
template<Permutable I, Sentinel<I> S, class T, class Proj = identity>
requires IndirectRelation<ranges::equal_to<>, projected<I, Proj>, const T*>
constexpr subrange<I> remove(I first, S last, const T& value, Proj proj = {});
template<ForwardRange R, class T, class Proj = identity>
requires Permutable<iterator_t<R>> &&
IndirectRelation<ranges::equal_to<>, projected<iterator_t<R>, Proj>, const T*>
constexpr safe_subrange_iterator_t<R>
remove(R&& r, const T& value, Proj proj = {});
template<Permutable I, Sentinel<I> S, class Proj = identity,
IndirectUnaryPredicate<projected<I, Proj>> Pred>
constexpr subrange<I> remove_if(I first, S last, Pred pred, Proj proj = {});
template<ForwardRange R, class Proj = identity,
IndirectUnaryPredicate<projected<iterator_t<R>, Proj>> Pred>
requires Permutable<iterator_t<R>>
constexpr safe_subrange_iterator_t<R>
remove_if(R&& r, Pred pred, Proj proj = {});
}
[...]
// 25.6.9 \[alg.unique\], unique
[...]
namespace ranges {
template<Permutable I, Sentinel<I> S, class Proj = identity,
IndirectRelation<projected<I, Proj>> C = ranges::equal_to<>>
constexpr subrange<I> unique(I first, S last, C comp = {}, Proj proj = {});
template<ForwardRange R, class Proj = identity,
IndirectRelation<projected<iterator_t<R>, Proj>> C = ranges::equal_to<>>
requires Permutable<iterator_t<R>>
constexpr safe_subrange_iterator_t<R>
unique(R&& r, C comp = {}, Proj proj = {});
}
[...]
// 25.7.1 \[alg.sort\], sorting
[...]
namespace ranges {
template<class I, class O> using partial\_sort\_copy\_result = copy\_result<I, O>;

template<InputIterator I1, Sentinel<I1> S1, RandomAccessIterator I2, Sentinel<I2> S2,
class Comp = ranges::less<>, class Proj1 = identity, class Proj2 = identity>
requires IndirectlyCopyable<I1, I2> && Sortable<I2, Comp, Proj2> &&
```

```

        IndirectStrictWeakOrder<Comp, projected<I1, Proj1>, projected<I2, Proj2>>
constexpr partial_sort_copy_result<I1, I2>
    partial_sort_copy(I1 first, S1 last, I2 result_first, S2 result_last,
        Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
template<InputRange R1, RandomAccessRange R2, class Comp = ranges::less<>,
    class Proj1 = identity, class Proj2 = identity>
requires IndirectlyCopyable<iterator_t<R1>, iterator_t<R2>> &&
    Sortable<iterator_t<R2>, Comp, Proj2> &&
    IndirectStrictWeakOrder<Comp, projected<iterator_t<R1>, Proj1>,
        projected<iterator_t<R2>, Proj2>>
constexpr partial_sort_copy_result<safe_iterator_t<R1>, safe_iterator_t<R2>>
    partial_sort_copy(R1&& r, R2&& result_r, Comp comp = {},
        Proj1 proj1 = {}, Proj2 proj2 = {});
}
[...]
```

// 25.7.4 [\[alg.partitions\]](#), partitions

```

[...]
```

```

namespace ranges {
    template<Permutable I, Sentinel<I> S, class Proj = identity,
        IndirectUnaryPredicate<projected<I, Proj>> Pred>
    constexpr subrange<I>
        partition(I first, S last, Pred pred, Proj proj = {});
    template<ForwardRange R, class Proj = identity,
        IndirectUnaryPredicate<projected<iterator_t<R>, Proj>> Pred>
    requires Permutable<iterator_t<R>>
    constexpr safe subrange_iterator_t<R>
        partition(R&& r, Pred pred, Proj proj = {});
}
[...]
```

```

namespace ranges {
    template<BidirectionalIterator I, Sentinel<I> S, class Proj = identity,
        IndirectUnaryPredicate<projected<I, Proj>> Pred>
    requires Permutable<I>
    subrange<I> stable_partition(I first, S last, Pred pred, Proj proj = {});
    template<BidirectionalRange R, class Proj = identity,
        IndirectUnaryPredicate<projected<iterator_t<R>, Proj>> Pred>
    requires Permutable<iterator_t<R>>
    safe subrange_iterator_t<R> stable_partition(R&& r, Pred pred, Proj proj = {});
}
[...]
```

2. Change 25.6.8 [\[alg.remove\]](#) as indicated:

```

[...]
```

```

namespace ranges {
    template<Permutable I, Sentinel<I> S, class T, class Proj = identity>
    requires IndirectRelation<ranges::equal_to<>, projected<I, Proj>, const T*>
    constexpr subrange<I> remove(I first, S last, const T& value, Proj proj = {});
    template<ForwardRange R, class T, class Proj = identity>
    requires Permutable<iterator_t<R>> &&
        IndirectRelation<ranges::equal_to<>, projected<iterator_t<R>, Proj>, const T*>
    constexpr safe subrange_iterator_t<R>
        remove(R&& r, const T& value, Proj proj = {});
    template<Permutable I, Sentinel<I> S, class Proj = identity,
        IndirectUnaryPredicate<projected<I, Proj>> Pred>
    constexpr subrange<I> remove_if(I first, S last, Pred pred, Proj proj = {});
    template<ForwardRange R, class Proj = identity,
        IndirectUnaryPredicate<projected<iterator_t<R>, Proj>> Pred>
    requires Permutable<iterator_t<R>>
    constexpr safe subrange_iterator_t<R>
        remove_if(R&& r, Pred pred, Proj proj = {});
}
[...]
```

-4- Returns: Let  $j$  be the end of the resulting range. Returns:

(4.2) —  $j$  for the overloads in namespace `std`, or

(4.2) —  $\{j, \text{last}\}$  for the overloads in namespace `ranges`.

[...]

3. Change 25.6.9 [\[alg.unique\]](#) as indicated:

```

[...]
```

```

namespace ranges {
    template<Permutable I, Sentinel<I> S, class Proj = identity,
        IndirectRelation<projected<I, Proj>> C = ranges::equal_to<>>
    constexpr subrange<I> unique(I first, S last, C comp = {}, Proj proj = {});
    template<ForwardRange R, class Proj = identity,
        IndirectRelation<projected<iterator_t<R>, Proj>> C = ranges::equal_to<>>
    requires Permutable<iterator_t<R>>
    constexpr safe subrange_iterator_t<R>
        unique(R&& r, C comp = {}, Proj proj = {});
}
[...]
```

-4- Returns: Let  $j$  be the end of the resulting range. Returns:

(4.2) —  $j$  for the overloads in namespace `std`, or

(4.2) —  $\{j, \text{last}\}$  for the overloads in namespace `ranges`.

[...]

4. Change 25.7.1.4 [partial.sort.copy] as indicated:

```
[...]
namespace ranges {
    template<InputIterator I1, Sentinel<I1> S1, RandomAccessIterator I2, Sentinel<I2> S2,
            class Comp = ranges::less<>, class Proj1 = identity, class Proj2 = identity>
    requires IndirectlyCopyable<I1, I2> && Sortable<I2, Comp, Proj2> &&
            IndirectStrictWeakOrder<Comp, projected<I1, Proj1>, projected<I2, Proj2>>
    constexpr partial_sort_copy_result<I1, I2>
        partial_sort_copy(I1 first, S1 last, I2 result_first, S2 result_last,
            Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
    template<InputRange R1, RandomAccessRange R2, class Comp = ranges::less<>,
            class Proj1 = identity, class Proj2 = identity>
    requires IndirectlyCopyable<iterator_t<R1>, iterator_t<R2>> &&
            Sortable<iterator_t<R2>, Comp, Proj2> &&
            IndirectStrictWeakOrder<Comp, projected<iterator_t<R1>, Proj1>,
                projected<iterator_t<R2>, Proj2>>
    constexpr partial_sort_copy_result<safe_iterator_t<R1>, safe_iterator_t<R2>>
        partial_sort_copy(R1&& r, R2&& result_r, Comp comp = {},
            Proj1 proj1 = {}, Proj2 proj2 = {});
}
```

[...]

-4- Returns:

(4.2) — result\_first + N for the overloads in namespace std, or

(4.2) — {last, result\_first + N} for the overloads in namespace ranges.

[...]

5. Change 25.7.4 [alg.partitions] as indicated:

```
[...]
namespace ranges {
    template<Permutable I, Sentinel<I> S, class Proj = identity,
            IndirectUnaryPredicate<projected<I, Proj>> Pred>
    constexpr subrange<I>
        partition(I first, S last, Pred pred, Proj proj = {});
    template<ForwardRange R, class Proj = identity,
            IndirectUnaryPredicate<projected<iterator_t<R>, Proj>> Pred>
    requires Permutable<iterator_t<R>>
    constexpr safe_subrange_iterator_t<R>
        partition(R&& r, Pred pred, Proj proj = {});
}
```

[...]

-7- Returns: Let  $i$  be an iterator such that  $E(*j)$  is true for every iterator  $j$  in  $[first, i)$  and false for every iterator  $j$  in  $[i, last)$ . Returns:

(7.2) —  $i$  for the overloads in namespace std, or

(7.2) —  $\{i, last\}$  for the overloads in namespace ranges.

[...]

```
[...]
namespace ranges {
    template<BidirectionalIterator I, Sentinel<I> S, class Proj = identity,
            IndirectUnaryPredicate<projected<I, Proj>> Pred>
    requires Permutable<I>
    subrange<I> stable_partition(I first, S last, Pred pred, Proj proj = {});
    template<BidirectionalRange R, class Proj = identity,
            IndirectUnaryPredicate<projected<iterator_t<R>, Proj>> Pred>
    requires Permutable<iterator_t<R>>
    safe_subrange_iterator_t<R> stable_partition(R&& r, Pred pred, Proj proj = {});
}
```

[...]

-11- Returns: Let  $i$  be an iterator such that for every iterator  $j$  in  $[first, i)$ ,  $E(*j)$  is true, and for every iterator  $j$  in the range  $[i, last)$ ,  $E(*j)$  is false. Returns:

(11.2) —  $i$  for the overloads in namespace std, or

(11.2) —  $\{i, last\}$  for the overloads in namespace ranges.

[...]

---

3187<sup>(i)</sup>. P0591R4 reverted DR 2586 fixes to scoped\_allocator\_adaptor::construct()

Section: 20.10.8.2 [allocator.uses.construction] Status: Ready Submitter: Jonathan Wakely Opened: 2019-02-14 Last modified: 2019-02-26

Priority: Not Prioritized

View other [active issues](#) in [allocator.uses.construction].

View all other [issues](#) in [allocator.uses.construction].

View all issues with [Ready](#) status.

#### Discussion:

[2586](#) fixed the value category in the uses-allocator checks done by `scoped_allocator_adaptor`. When we made that use `uses_allocator_construction_args` we reintroduced the problem, because that function has the same bug.

```
#include <memory>

struct X {
    using allocator_type = std::allocator<X>;
    X(std::allocator_arg_t, allocator_type&&) { }
    X(const allocator_type&) { }
};

int main() {
    std::allocator<X> a;
    std::make_obj_using_allocator<X>(a);
}
```

This will fail to compile, because `uses_allocator_construction_args` will check `is_constructible` using an rvalue allocator, but then return `tuple<allocator_arg_t, const allocator<X>&>` containing an lvalue allocator. Those args cannot be used to construct an X.

[2019-02; Kona Wednesday night issue processing]

Status to Ready

#### Proposed resolution:

This wording is relative to [N4800](#).

1. Change 20.10.8.2 [\[allocator.uses.construction\]](#) as indicated:

[Drafting Note: Arguably the `uses_allocator` specialization should also use `const Alloc&` but in practice that doesn't matter, except for even more contrived cases than the very contrived example above.]

```
template <class T, class Alloc, class... Args>
auto uses_allocator_construction_args(const Alloc& alloc, Args&&... args) -> see below;
```

[...]

-5- Returns: A tuple value determined as follows:

1. (5.1) — If `uses_allocator_v<T, Alloc>` is false and `is_constructible_v<T, Args...>` is true, return `forward_as_tuple(std::forward<Args>(args)...) .`
2. (5.2) — Otherwise, if `uses_allocator_v<T, Alloc>` is true and `is_constructible_v<T, allocator_arg_t, const Alloc&, Args...>` is true, return  

```
tuple<allocator_arg_t, const Alloc&, Args&&...>(
    allocator_arg, alloc, std::forward<Args>(args)...) .
```
3. (5.3) — Otherwise, if `uses_allocator_v<T, Alloc>` is true and `is_constructible_v<T, Args..., const Alloc&>` is true, return `forward_as_tuple(std::forward<Args>(args)..., alloc) .`
4. (5.4) — Otherwise, the program is ill-formed.

[...]

---

## [3191<sup>\(i\)</sup>](#). `std::ranges::shuffle` synopsis does not match algorithm definition

Section: 25.6.13 [\[alg.random.shuffle\]](#) Status: [Tentatively Ready](#) Submitter: Christopher Di Bella Opened: 2019-03-02 Last modified: 2019-06-16

Priority: 1

View all other [issues](#) in [alg.random.shuffle].

View all issues with [Tentatively Ready](#) status.

#### Discussion:

25.4 [\[algorithm.syn\]](#) declares `std::ranges::shuffle` like so:

```
namespace ranges {
    template<RandomAccessIterator I, Sentinel<I> S, class Gen>
        requires Permutable<I> &&
            UniformRandomBitGenerator<remove_reference_t<Gen>> &&
            ConvertibleTo<invoke_result_t<Gen&, iter_difference_t<I>>>
    I shuffle(I first, S last, Gen&& g);

    template<RandomAccessRange R, class Gen>
```

```

    requires Permutable<iterator_t<R>> &&
           UniformRandomBitGenerator<remove_reference_t<Gen>> &&
           ConvertibleTo<invoke_result_t<Gen&&, iter_difference_t<iterator_t<R>>>>
safe_iterator_t<R> shuffle(R&& r, Gen&& g);
}

```

25.6.13 [\[alg.random.shuffle\]](#) defines the algorithm like so:

```

namespace ranges {
    template<RandomAccessIterator I, Sentinel<I> S, class Gen>
        requires Permutable<I> &&
             UniformRandomBitGenerator<remove_reference_t<Gen>>
    I shuffle(I first, S last, Gen&& g);

    template<RandomAccessRange R, class Gen>
        requires Permutable<iterator_t<R>> &&
             UniformRandomBitGenerator<remove_reference_t<Gen>>
    safe_iterator_t<R> shuffle(R&& r, Gen&& g);
}

```

Notice the missing `ConvertibleTo` requirements in the latter. Looking at the [Ranges TS](#), `[alg.random.shuffle]` includes this requirement, albeit in the Ranges TS-format.

[2019-03-03; Daniel comments]

Given that the accepted proposal [P0896R4](#) voted in San Diego did contain the same error I decided to open this issue instead of submitting an editorial change request.

[2019-03-05 Priority set to 1 after reflector discussion]

Casey: The correct fix here is to remove the `ConvertibleTo` requirement from the header synopsis. `UniformRandomBitGenerators` have integral result types, and the core language guarantees that all integral types are convertible to all other integral types. We don't need to validate the core language in the associated constraints of `ranges::shuffle`.

**Previous resolution [SUPERSEDED]:**

This wording is relative to [N4800](#).

1. Change 25.6.13 [\[alg.random.shuffle\]](#) as indicated:

```

[...]
namespace ranges {
    template<RandomAccessIterator I, Sentinel<I> S, class Gen>
        requires Permutable<I> &&
             UniformRandomBitGenerator<remove_reference_t<Gen>> &&
             ConvertibleTo<invoke_result_t<Gen&&, iter_difference_t<I>>>
    I shuffle(I first, S last, Gen&& g);
    template<RandomAccessRange R, class Gen>
        requires Permutable<iterator_t<R>> &&
             UniformRandomBitGenerator<remove_reference_t<Gen>> &&
             ConvertibleTo<invoke_result_t<Gen&&, iter_difference_t<iterator_t<R>>>
    safe_iterator_t<R> shuffle(R&& r, Gen&& g);
}

```

[2019-03-05 Updated proposed wording according to Casey's suggestion]

[2019-06-16 Set to "Tentatively Ready" after five positive votes on the reflector.]

**Proposed resolution:**

This wording is relative to [N4800](#).

1. Change 25.4 [\[algorithm.syn\]](#) as indicated:

```

// 25.6.13 \[alg.random.shuffle\], shuffle
[...]
namespace ranges {
    template<RandomAccessIterator I, Sentinel<I> S, class Gen>
        requires Permutable<I> &&
             UniformRandomBitGenerator<remove_reference_t<Gen>> &&
             ConvertibleTo<invoke_result_t<Gen&&, iter_difference_t<I>>>
    I shuffle(I first, S last, Gen&& g);

    template<RandomAccessRange R, class Gen>
        requires Permutable<iterator_t<R>> &&
             UniformRandomBitGenerator<remove_reference_t<Gen>> &&
             ConvertibleTo<invoke_result_t<Gen&&, iter_difference_t<iterator_t<R>>>
    safe_iterator_t<R> shuffle(R&& r, Gen&& g);
}
[...]

```

**3196<sup>(1)</sup>. `std::optional<T>` is ill-formed if `T` is an array**

**Section:** 20.6.3 [\[optional.optional\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2019-03-18 **Last modified:** 2019-06-16

**Priority:** 0

**View other [active issues](#)** in `[optional.optional]`.

**View all other [issues](#)** in `[optional.optional]`.

View all issues with [Tentatively Ready](#) status.

Discussion:

20.6.3 [\[optional.optional\]](#) appears to allow arrays:

$\tau$  shall be an object type other than `cv in_place_t` or `cv nullopt_t` and shall satisfy the *Cpp17Destructible* requirements (Table 30).

But instantiating it fails, because `value_or` attempts to return  $\tau$  by value, which isn't possible for an array type.

Existing practice seems to be to reject array types. Libstdc++ and libc++ give an error for the signature of `value_or`, and MSVC fails a static assert saying the type needs to be destructible (which is misleading, because `int[2]` is destructible, but either way it's ill-formed).

Previous resolution [SUPERSEDED]:

This wording is relative to [N4810](#).

1. Modify 20.6.3 [\[optional.optional\]](#) as indicated:

-3-  $\tau$  shall be an **non-array** object type other than `cv in_place_t` or `cv nullopt_t` and shall satisfy the *Cpp17Destructible* requirements (Table 30).

[2019-03-26 Marshall provides updated resolution based on reflector discussion]

[2019-06-16 Moved to "Tentatively Ready" based on five positive votes on the reflector]

Proposed resolution:

This wording is relative to [N4810](#).

1. In 16.5.3.1 [\[utility.arg.requirements\]](#), edit Table 30 — "*Cpp17Destructible* requirements" as indicated:

| Table 30 — <i>Cpp17Destructible</i> requirements                                             |                                                                                  |
|----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Expression                                                                                   | Post-condition                                                                   |
| <code>u.~T()</code>                                                                          | All resources owned by <code>u</code> are reclaimed, no exception is propagated. |
| <b>[Note: Array types and non-object types are not <i>Cpp17Destructible</i>. — end note]</b> |                                                                                  |

1. Modify 20.6.3 [\[optional.optional\]](#) as indicated:

-3-  $\tau$  shall be a **object** type other than `cv in_place_t` or `cv nullopt_t` **and shall satisfy that meets** the *Cpp17Destructible* requirements (Table 30).

1. Modify 20.7.3 [\[variant.variant\]](#) as indicated:

-2- All types in Types shall **be (possibly cv-qualified) object types that are not arrays meet the *Cpp17Destructible* requirements (Table 30).**

[3198<sup>\(i\)</sup>](#). **Bad constraint on `std::span::span()`**

**Section:** 22.7.3.2 [\[span.cons\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Lars Gullik Bjønnes **Opened:** 2019-04-03 **Last modified:** 2019-06-16

**Priority:** Not Prioritized

View all other [issues](#) in [\[span.cons\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

It seems that this was left out of [P1089](#). The constraint on `span()` (22.7.3.2 [\[span.cons\]](#)) in the current draft is:

*Constraints:* `Extent <= 0` is true.

This does not seem to make much sense.

The proposal is to change the constraint to be:

*Constraints:* `Extent == dynamic_extent || Extent == 0` is true.

[2019-06-09; Priority to 0 and Status to Tentatively Ready after five positive votes on the reflector.]

Proposed resolution:

This wording is relative to [N4810](#).

1. Modify 22.7.3.2 [\[span.cons\]](#) as indicated:

`constexpr span() noexcept;`

-1- *Constraints:* `Extent <= dynamic_extent || Extent == 0` is true.

[3199<sup>\(i\)</sup>](#). **`istream >> bitset<0>` fails**



**Section:** 20.9.4 [\[bitset.operators\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Davis Herring **Opened:** 2019-04-05 **Last modified:** 2019-06-16

**Priority:** Not Prioritized

**View all other [issues](#)** in [\[bitset.operators\]](#).

**View all issues with [Tentatively Ready](#) status.**

**Discussion:**

From a [StackOverflow question](#):

[\[bitset.operators\]/5.1](#) says that extracting a `bitset<0>` stops after reading 0 characters. /6 then says that, since no characters were stored, `failbit` is set.

[2019-06-09; Priority to 0 and Status to Tentatively Ready after five positive votes on the reflector.]

**Proposed resolution:**

This wording is relative to [N4810](#).

1. Modify 20.9.4 [\[bitset.operators\]](#) as indicated:

```
template<class charT, class traits, size_t N>
basic_istream<charT, traits>&
operator>>(basic_istream<charT, traits>& is, bitset<N>& x);

[...]
```

-6- If `N > 0` and no characters are stored in `str`, calls `is.setstate(ios_base::failbit)` (which may throw `ios_base::failure` (29.5.5.4 [\[iostate.flags\]](#))).

---

## [3202<sup>\(i\)</sup>](#). P0318R1 was supposed to be revised

**Section:** 20.14.1 [\[functional.syn\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2019-04-23 **Last modified:** 2019-06-16

**Priority:** 0

**View all issues with [Tentatively Ready](#) status.**

**Discussion:**

[P0318R1](#) was discussed in Batavia and the requested changes were made in D0318R2. In San Diego the R1 paper was voted into the WP, despite not having the requested changes. There were also changes to D0318R2 suggested on the reflector, which are incorporated below.

[2019-04-30 Priority to 0 and Status to Tentatively Ready after six positive votes on the reflector.]

**Proposed resolution:**

This wording is relative to [N4810](#).

1. Modify 20.14.1 [\[functional.syn\]](#), header `<functional>` synopsis, as indicated:

```
template<class T> struct unwrap_reference;
template<class T> using unwrap_reference_t = typename unwrap_reference<T>::type;
template<class T> struct unwrap_ref_decay : unwrap_reference<decay_t<T>> {};
template<class T> using unwrap_ref_decay_t = typename unwrap_ref_decay<T>::type;
```

2. Modify 20.14.5.6 [\[refwrap.unwrapref\]](#) as indicated:

```
template<class T>
struct unwrap_reference;
```

-1- If `T` is a specialization [...]

```
template<class T>
struct unwrap_ref_decay;
```

-2- The member typedef `type` of `unwrap_ref_decay<T>` denotes the type `unwrap_reference_t<decay_t<T>>`.

---

## [3206<sup>\(i\)</sup>](#). `year_month_day` conversion to `sys_days` uses not-existing member function

**Section:** 27.8.14.2 [\[time.cal.ymd.members\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tomasz Kamiński **Opened:** 2019-05-19 **Last modified:** 2019-06-16

**Priority:** 0

**View all issues with [Tentatively Ready](#) status.**

**Discussion:**

The current specification of the `year_month_day` conversion function to `sys_days`, uses the day member function on the `sys_days` (a.k.a. `time_point<system_clock, days>`), that does not exist.

In 27.8.14.2 [\[time.cal.ymd.members\]](#) p18, the expression `sys_days{y_/m_/last}.day()` is ill-formed:

[...] Otherwise, if `y_.ok()` && `m_.ok()` is true, returns a `sys_days` which is offset from `sys_days{y_/m_/last}` by the number of days `d_` is offset from `sys_days{y_/m_/last}.day()`. Otherwise the value returned is unspecified.

[2019-06-08; Priority to 0 and Status to Tentatively Ready after six positive votes on the reflector.]

#### Proposed resolution:

This wording is relative to [N4810](#).

1. Modify 27.8.14.2 [\[time.cal.ymd.members\]](#) as indicated:

```
constexpr operator sys_days() const noexcept;
```

-18- *Returns:* If `ok()`, returns a `sys_days` holding a count of days from the `sys_days` epoch to `*this` (a negative value if `*this` represents a date prior to the `sys_days` epoch). Otherwise, if `y_.ok()` && `m_.ok()` is true, returns `sys_days{y_/m_/1d} + (d_ - 1d) * sys_days` which is offset from `sys_days{y_/m_/last}` by the number of days `d_` is offset from `sys_days{y_/m_/last}.day()`. Otherwise the value returned is unspecified.

### [3208<sup>\(i\)</sup>](#). Boolean's expression requirements are ordered inconsistently

**Section:** 23.7 [\[iterator.range\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2019-06-06 **Last modified:** 2019-06-16

**Priority:** 0

**View other** [active issues](#) in [\[iterator.range\]](#).

**View all other** [issues](#) in [\[iterator.range\]](#).

**View all issues with** [Tentatively Ready](#) status.

#### Discussion:

For consistency of presentation, we should group and order the `&&` and `||` expression requirements similarly to the `==` and `!=` expression requirements. Note that the suggested change is not quite editorial: evaluation of requirements for satisfaction has short-circuiting behavior, so the declaration order of requirements is normatively significant in general.

[2019-06-13; Priority to 0 and Status to Tentatively Ready after seven positive votes on the reflector.]

#### Proposed resolution:

This wording is relative to [N4810](#).

1. Modify 18.5.2 [\[concept.boolean\]](#) as indicated:

```
[...]
{ b1 } -> ConvertibleTo<bool>;
{ !b1 } -> ConvertibleTo<bool>;
{ b1 && a } -> Same<bool>;
{ b1 || a } -> Same<bool>;
{ b1 && b2 } -> Same<bool>;
{ b1 && a } -> Same<bool>;
{ a && b2 } -> Same<bool>;
{ b1 || b2 } -> Same<bool>;
{ b1 || a } -> Same<bool>;
{ a || b2 } -> Same<bool>;
{ b1 == b2 } -> ConvertibleTo<bool>;
{ b1 == a } -> ConvertibleTo<bool>;
{ a == b2 } -> ConvertibleTo<bool>;
{ b1 != b2 } -> ConvertibleTo<bool>;
{ b1 != a } -> ConvertibleTo<bool>;
{ a != b2 } -> ConvertibleTo<bool>;
};
```

### [3209<sup>\(i\)</sup>](#). Expression in `year::ok()` returns clause is ill-formed

**Section:** 27.8.5.2 [\[time.cal.year.members\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tomasz Kamiński **Opened:** 2019-05-28 **Last modified:** 2019-06-16

**Priority:** 0

**View all issues with** [Tentatively Ready](#) status.

#### Discussion:

The expression in the *Returns* element of `year::ok` in [\[time.cal.year.members\]](#) p18:

```
min() <= y_ && y_ <= max()
```

is ill-formed, as it attempts to compare type `short` (type of `y_ member`) with type `year` (type of `year::min()` and `year::max()`).

[2019-06-13; Priority to 0 and Status to Tentatively Ready after seven positive votes on the reflector.]

#### Proposed resolution:

This wording is relative to [N4810](#).

1. Modify 27.8.5.2 [\[time.cal.year.members\]](#) as indicated:

```
constexpr bool ok() const noexcept;
```

-18- Returns: `min()_y_ <= y_ && y_ <= max()_y_`.